

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ  
ФЕДЕРАЦИИ  
Федеральное государственное бюджетное образовательное учреждение  
высшего образования  
**«КУБАНСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ»**  
**(ФГБОУ ВО «КубГУ»)**

**Факультет компьютерных технологий и прикладной математики**  
**Кафедра вычислительных технологий**

**ИНДИВИДУАЛЬНОЕ ЗАДАНИЕ №2**  
**Дисциплина: «Оценка сложности алгоритмов»**

Работу выполнил: \_\_\_\_\_ Костров А.А.

Направление подготовки: 02.03.02 Фундаментальная информатика и  
информационные технологии

Преподаватель: \_\_\_\_\_ Яхонтов А.А.

*Тема.* NP-полные задачи.

*Цель.* Определить принадлежность задачи к классу NP. Определить принадлежность задачи к классу NP-полных или NP-трудных задач. Определить, является ли задача NP-полной в сильном смысле. Изучить приближённые методы решения задачи.

*Задача планирования работ.* Пусть у нас есть набор работ, и мы знаем время, необходимое для завершения каждой из них:  $t_1, t_2, \dots, t_N$ ; сроки:  $d_1, d_2, \dots, d_N$ , к которым эти работы должны быть обязательно завершены; а также штрафы:  $p_1, p_2, \dots, p_N$ , которые будут наложены при невыполнении каждой работы в установленные сроки. Задача оптимизации требует установить порядок работ, минимизирующий накладываемые штрафы. Задача принятия решений: есть ли порядок работ, при котором величина штрафа будет не больше  $P$ .

*Принадлежность к классу NP.* Задача принятия решений принадлежит классу NP.

Недетерминированный двухэтапный процесс решения:

1. Недетерминированный этап (угадывание): недетерминированно (случайно) выбирается полная перестановка (порядок) всех  $N$  работ.

Формат выхода недетерминированного шага: список индексов работ в порядке их выполнения, например:  $[3, 1, 4, 2, \dots, N]$  (все индексы от 1 до  $N$  ровно один раз). Это полное решение задачи.

2. Детерминированный этап проверки (верификация): вычисляем время завершения для каждой работы в предложенном порядке:

$current\_time = 0$

Для каждой работы  $j$  в порядке:

$current\_time += t[j]$

$C[j] = current\_time$

if  $C[j] > d[j]$ :

$штраф += p[j]$

Проверяем, что суммарный штраф  $\leq P$ .

Проверка выполняется за  $O(N)$  (линейный проход). Поскольку проверка полиномиальна, задача принадлежит классу NP.

*Принадлежность к NP-полным или NP-трудным задачам.* Задача оптимизации NP-трудная. Задача принятия решений NP-полная.

NP-трудность показывается через полиномиальное сведение от другой NP-полной задачи: *Subset Sum*.

Subset Sum (NP-полная задача):

- Дано множество положительных целых чисел  $\{a_1, a_2, \dots, a_n\}$  и целевое значение  $T$ .
- Вопрос: существует ли подмножество, сумма элементов которого равно  $T$ ?

Сведение в задачу планирования (экземпляр строится за полиномиальное время):

Создаём  $n + 1$  работ:

1.  $n$  основных работ (соответствуют числам  $a_i$ ):
  - $t_i = a_i$  (время выполнения)
  - $d_i = T$  (дедлайн)
  - $p_i = 1$  (штраф, любое одно и то же положительное число для всех работ)
2. Одна дополнительная работа:
  - $t_0 = 1$
  - $d_0 = T + 1$
  - $p_0 = 0$  (или очень большое число, чтобы работа всегда выполнялась вовремя)

Целевое значение штрафа  $K = n$  (или любое число, большее  $n$ ).

Если Subset Sum даёт ответ YES, то существует расписание со штрафом  $\leq n$ :

- Берём подмножество  $X$ , сумма которого =  $T$ .
- Сначала выполняем все работы из  $X \rightarrow$  их суммарное время =  $T$ , все завершаются ровно в момент  $T \rightarrow$  вовремя (штраф 0).
- Затем выполняем дополнительную работу 0 (время 1)  $\rightarrow$  завершается в  $T+1 \leq d_0$ .
- В конце выполняем оставшиеся  $n - |X|$  работ  $\rightarrow$  они все опаздывают  $\rightarrow$  штраф =  $(n - |X|) \times 1$ .
- Итоговый штраф =  $n - |X| \leq n$ .

*NP-полнота в сильном смысле.* Задача является NP-полной в слабом смысле. Для задачи существует псевдополиномиальный алгоритм (сложность  $O(n \cdot \sum w_j)$  или  $O(n \cdot \sum t_j)$  в зависимости от реализации).

*Алгоритмы решения.* Точный алгоритм: перебор всех перестановок —  $O(N! \times N)$ .

Эвристические алгоритмы:

1. EDD (Earliest Due Date) — сортировка по возрастанию дедлайнов.

Сложность:  $O(N \log N)$ .

2. Greedy по убыванию штрафа  $p_i$ . Сложность:  $O(N \log N)$ .

3. Локальный поиск на перестановках. Сложность:  $O(N \times Iterations)$ .

## Реализация на Python.

```
import itertools
import random

def calculate_penalty(schedule):
    current_time = 0.0
    total_penalty = 0.0
    for _, t, d, p in schedule:
        current_time += t
        if current_time > d:
            total_penalty += p
    return total_penalty

def exact_brute_force(jobs):
    min_penalty = float('inf')
    best_order = None
    for perm in itertools.permutations(jobs):
        pen = calculate_penalty(perm)
        if pen < min_penalty:
            min_penalty = pen
            best_order = [j[0] for j in perm]
    return best_order, min_penalty

def heuristic_edd(jobs):
    sorted_jobs = sorted(jobs, key=lambda x: x[2])
    order = [j[0] for j in sorted_jobs]
    pen = calculate_penalty(sorted_jobs)
    return order, pen

def heuristic_high_penalty_first(jobs):
    sorted_jobs = sorted(jobs, key=lambda x: x[3], reverse=True)
    order = [j[0] for j in sorted_jobs]
    pen = calculate_penalty(sorted_jobs)
    return order, pen

def heuristic_local_search(jobs, iterations=1000):
    current = list(jobs)
    best_penalty = calculate_penalty(current)
    best_schedule = [j[0] for j in current]

    for _ in range(iterations):
        i, j = random.sample(range(len(current)), 2)
        current[i], current[j] = current[j], current[i]
        pen = calculate_penalty(current)
        if pen < best_penalty:
            best_penalty = pen
            best_schedule = [x[0] for x in current]
        else:
            current[i], current[j] = current[j], current[i]
    return best_schedule, best_penalty

if __name__ == "__main__":
    jobs = [
        (1, 2, 5, 10),
        (2, 3, 6, 20),
        (3, 1, 4, 5),
        (4, 4, 10, 15)
    ]

    print("Brute Force:", exact_brute_force(jobs))
    print("EDD:", heuristic_edd(jobs))
    print("High Penalty First:", heuristic_high_penalty_first(jobs))
    print("Local Search:", heuristic_local_search(jobs))
```